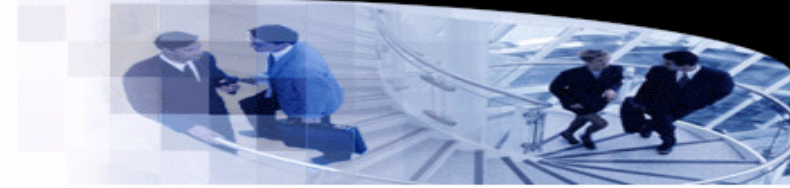




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

Lecture 17. Syntax-Directed Translation

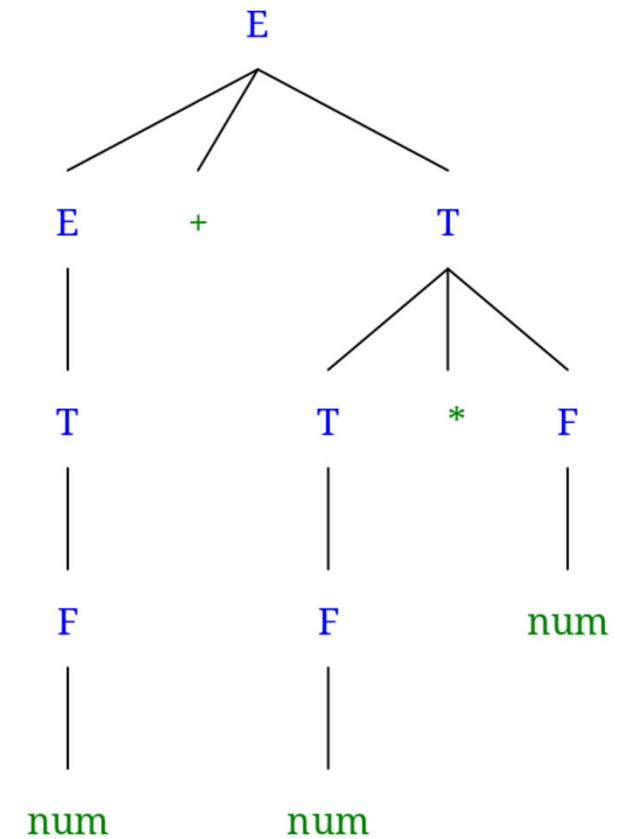
袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Annotated Syntax Tree

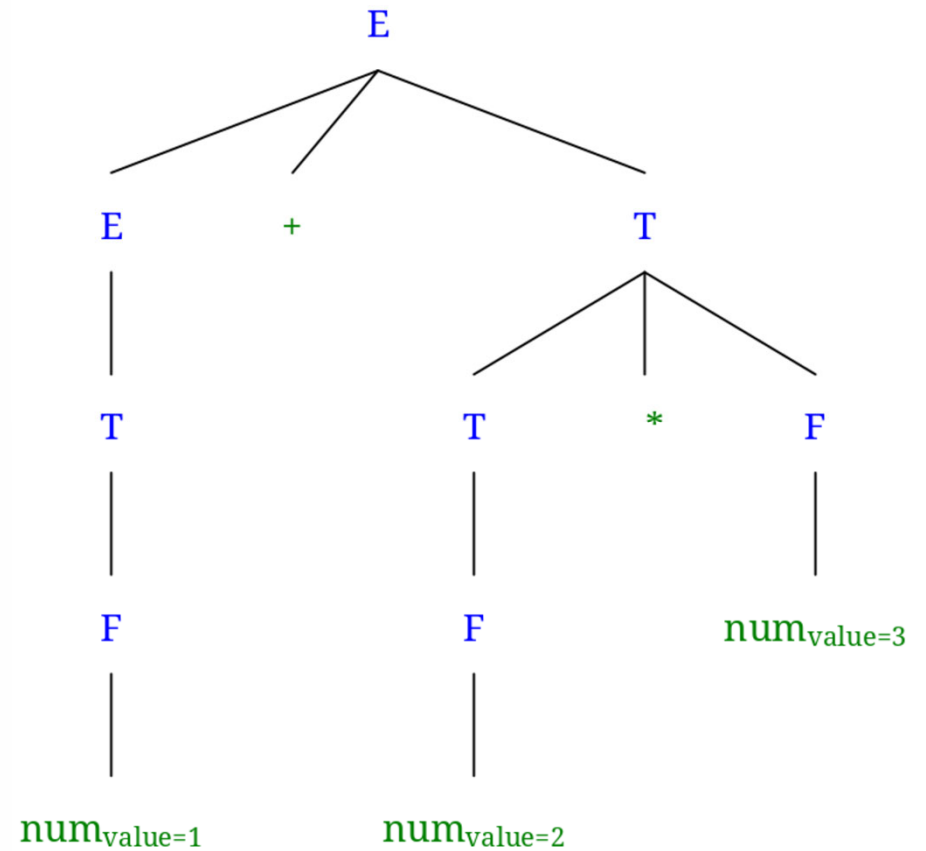
- Algorithm of semantic evaluation
 - Parse the input string to obtain a syntax tree.



Annotated Syntax Tree

- Algorithm of semantic evaluation

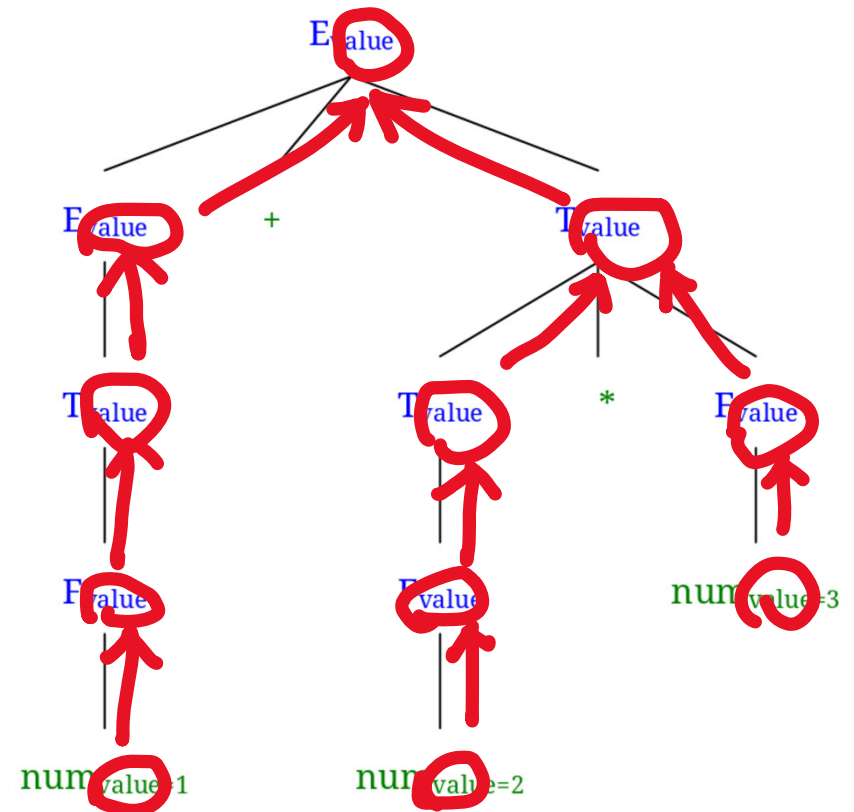
- Parse the input string to obtain a syntax tree.
- Get attributes from the lexer.



Annotated Syntax Tree

- Algorithm of semantic evaluation

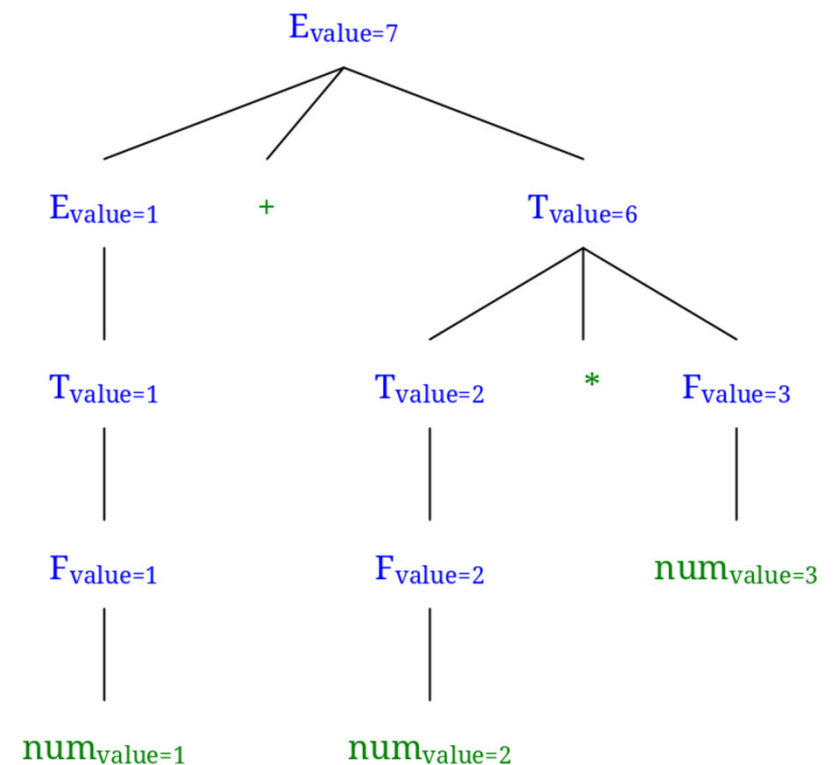
- Parse the input string to obtain a syntax tree.
- Get attributes from the lexer.
- Generate a dependency graph according to evaluation rules.



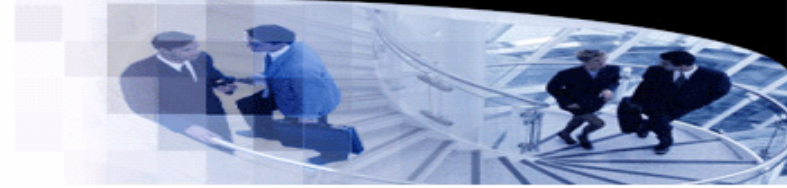
Annotated Syntax Tree

- Algorithm of semantic evaluation

- Parse the input string to obtain a syntax tree.
- Get attributes from the lexer.
- Generate a dependency graph according to evaluation rules.
- Calculate all attributes following the dependency graph.



Syntax-Directed Translation



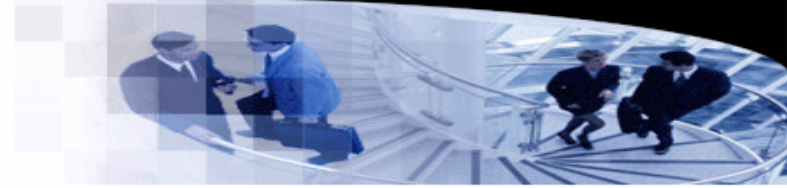
- **Annotated syntax tree**

- In addition to the whole syntax tree, the decoration requires extra space to store attributes and the dependency graph.
- However, some attributes can be dropped after being used.

- **Syntax-directed translation**

- As evaluation rules are associated to productions, it is possible to evaluate attribute during parsing.
- Therefore, the evaluation/translation is driven by the parser.
- Whenever the parser performs an action of derivation or reduction, the associated rules are used to calculate attributes.

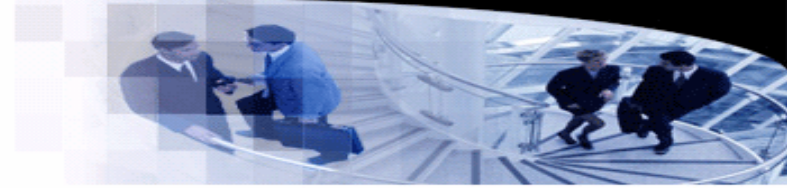
Syntax-Directed Translation



- **Syntax-directed translation schemes**

- A syntax-directed translation scheme is a set of productions that have semantic actions embedded inside.
- $A \rightarrow \alpha_1\{Action_1\}\alpha_2\{Action_2\} \dots \alpha_n\{Action_n\}$
- It can be converted into a postfix form.
 - $A \rightarrow \alpha_1 M_1 \alpha_2 M_2 \dots \alpha_n\{Action_n\}$
 - $M_1 \rightarrow \epsilon\{Action_1\}$
 - $M_2 \rightarrow \epsilon\{Action_2\}$
 - ...
 - $M_{n-1} \rightarrow \epsilon\{Action_{n-1}\}$

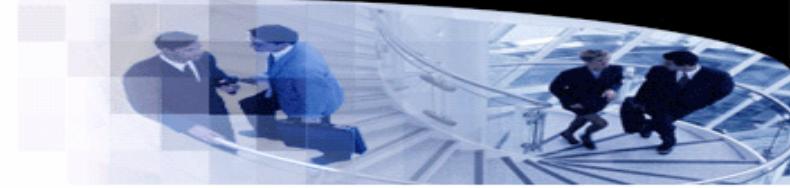
Syntax-Directed Translation



- Bottom-up semantic evaluation

- The parser maintains a **semantic stack** that is synchronous to the **symbol stack**.
- For each symbol in the symbol stack, its attributes are stored in the corresponding cell of the semantic stack.
- The attributes of terminals are given by the scanner.
- The attributes of nonterminals are evaluated before being pushed to the stack.
- When the parser chooses a production to reduce, the corresponding translation action is invoked.

Syntax-Directed Translation

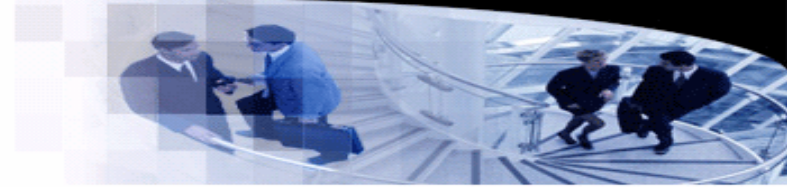


- Bottom-up semantic evaluation
 - Example:

Productions	Semantic Rules	Semantic Actions
$E \rightarrow E^1 + T$	$E.value = E^1.value + T.value$	$\{ss[newTop].value := ss[top + 2].value + ss[top].value;\}$
$E \rightarrow T$	$E.value = T.value$	$\{E.value := T.value;\}$
$T \rightarrow T^1 * F$	$T.value = T^1.value * F.value$	$\{\$$.value := \$1.value * \$3.value;\}$
$T \rightarrow F$	$T.value = F.value$	$\{T.value := F.value;\}$
$F \rightarrow (E)$	$F.value = E.value$	$\{F.value := E.value;\}$
$F \rightarrow num$	$F.value = num.value$	$\{F.value := num.value;\}$

- ss is the semantic stack. top indicates the top location of the semantic stack.
- $$$, \$1, \$2, \dots$ are yacc-style stack locations. $$$$ is the location of new top location of the stack. $\$i$ is the location of the i^{th} symbol of the production in the stack.
- For convenience, we use nonterminals (e.g., E and E^1) to represent their locations in the semantic stack.

Syntax-Directed Translation

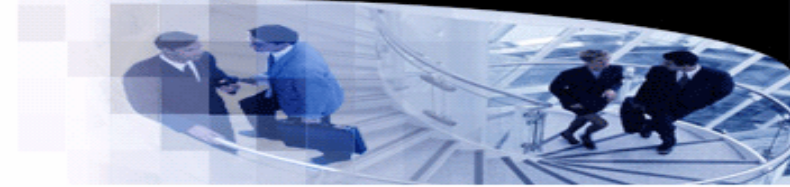


- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

	+	num	*	num	\$
--	---	-----	---	-----	----

Symbol Stack	Semantic Stack
num	1
\$	

Syntax-Directed Translation



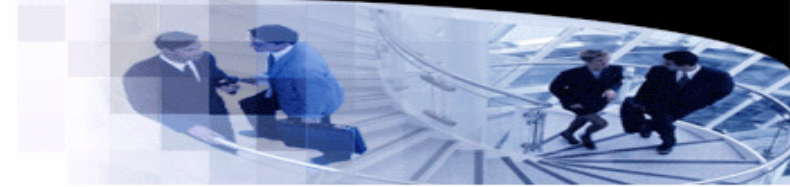
- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

	+	num	*	num	\$
--	---	-----	---	-----	----

Symbol Stack	Semantic Stack
F	1
\$	

$\{F.value := num.value;\}$

Syntax-Directed Translation



- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

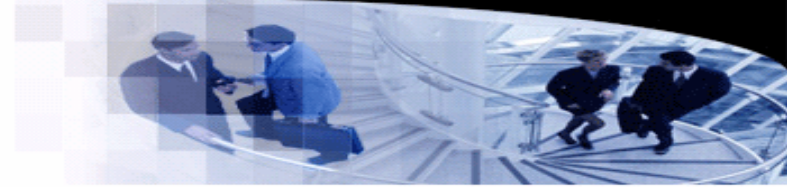
	+	num	*	num	\$
--	---	-----	---	-----	----

Symbol Stack	Semantic Stack
E	1
\$	

$\{T.value := F.value;\}$

$\{E.value := T.value;\}$

Syntax-Directed Translation

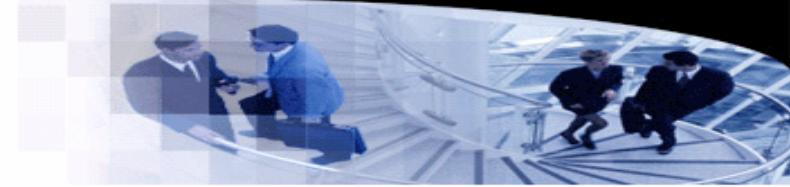


- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

			*	num	\$
--	--	--	---	-----	----

Symbol Stack	Semantic Stack
num	2
+	
E	1
\$	

Syntax-Directed Translation



- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

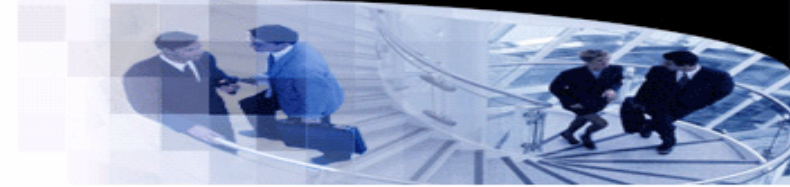
			*	num	\$
--	--	--	---	-----	----

Symbol Stack	Semantic Stack
T	2
+	
E	1
\$	

$\{F.value := num.value;\}$

$\{T.value := F.value;\}$

Syntax-Directed Translation

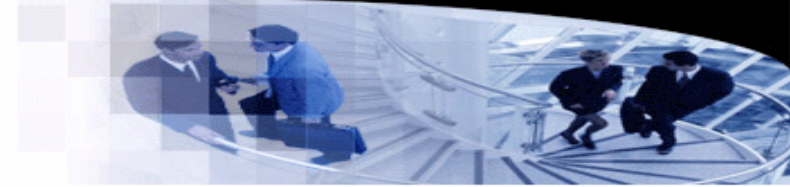


- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

					\$
--	--	--	--	--	----

Symbol Stack	Semantic Stack
num	3
*	
T	2
+	
E	1
\$	

Syntax-Directed Translation



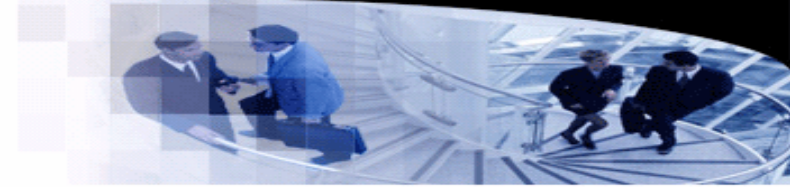
- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

					\$
--	--	--	--	--	----

Symbol Stack	Semantic Stack
F	3
*	
T	2
+	
E	1
\$	

$\{F.value := num.value;\}$

Syntax-Directed Translation



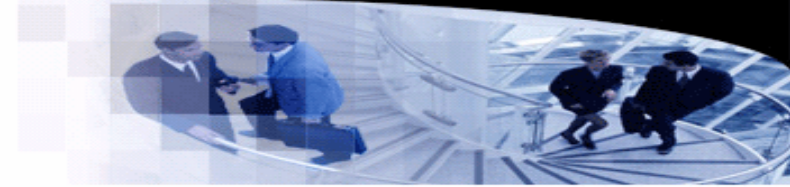
- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

					\$
--	--	--	--	--	----

Symbol Stack	Semantic Stack
T	6
+	
E	1
\$	

$\{T.value := T^1.value * F.value;\}$

Syntax-Directed Translation



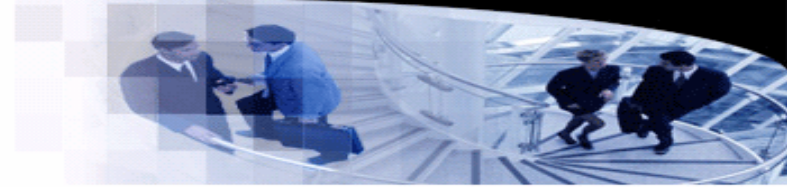
- Bottom-up semantic evaluation
 - Example: $1 + 2 * 3$

					\$
--	--	--	--	--	----

Symbol Stack	Semantic Stack
E	7
\$	

$\{E.value := E^1.value + T.value;\}$

Syntax-Directed Translation



- **Synthesized attributes**

- Given a production $A \rightarrow \alpha$ and its evaluation rule $A.x = f(y_1, y_2, \dots, y_n)$, x is called a synthesized attribute if $y_1, y_2, \dots, y_n$ are all attributes of the symbols in α .

$$E \rightarrow E^1 + T$$

$$E.value = E^1.value + T.value$$

- In bottom-up parsers, synthesized attributes can be calculated naturally.

- **Inherited attributes**

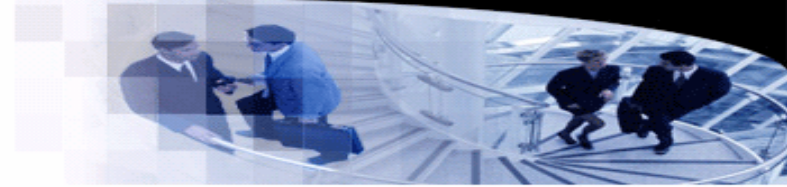
- Given an evaluation rule $A.x = f(y_1, y_2, \dots, y_n)$, x is called an inherited attribute if $y_1, y_2, \dots, y_n$ are attributes of A 's parent node or sibling nodes.

$$R \rightarrow^* R^1$$

$$R^1.spec = R.spec$$
$$R^1.type = pointer(R.type)$$

- It is non-trivial to calculate inherited attributes in bottom-up parsers.

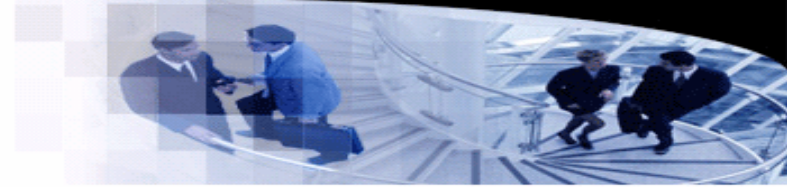
Syntax-Directed Translation



- Calculating inherited attributes in bottom-up parsers
 - The challenge of calculating inherited attributes is that their depended attributes may not occur in the stack.
 - However, sometimes we can “guess” the location of the original depended attribute in the stack by analyzing the grammar.
 - Example:

Productions	Semantic Rules
$D \rightarrow TL$	$L.t = T.t$
$T \rightarrow int$	$T.t = integer$
$L \rightarrow L^1 * id$	$id.t = L.t$ $L^1.t = L.t$
$L \rightarrow id$	$id.t = L.t$

Syntax-Directed Translation



- Calculating inherited attributes in bottom-up parsers

- For some grammars, the depended value may not be at the fixed location of the stack.
- Therefore, the value cannot be “guessed”.
- In this case, we can modify the grammar to adjust the location of the depended value.

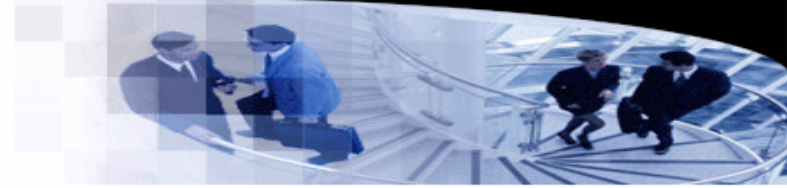
Productions	Semantic Rules
$S \rightarrow aBC$	$C.i = B.s$
$S \rightarrow aBAC$	$C.i = B.s$
$C \rightarrow c$	$C.s = f(C.i)$

Productions	Semantic Rules	Semantic Actions
$S \rightarrow aBC$	$C.i = B.s$	
$S \rightarrow aBAMC$	$C.i = M.s$ $M.i = B.s$	
$C \rightarrow c$	$C.s = f(C.i)$	$\{\$$.s = f(\$0.i)\}$
$M \rightarrow \epsilon$	$M.s = M.i$	

- Example:

- The modified grammar is presented on the right-hand side.
- The depended value of $C.i$ is at the location of $\$0$, whichever production of S was chosen.

Syntax-Directed Translation



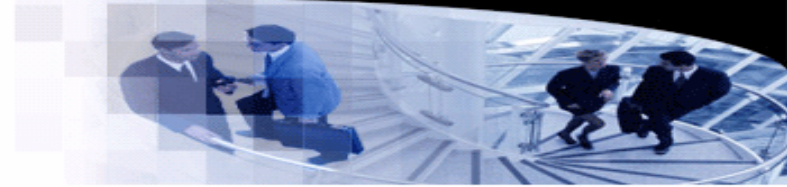
- Translation via recursive descent parsers

- Recursive descent parser, however, can calculate synthesized attributes and some inherited attributes.
- For each symbol A , we design a corresponding parsing function A .
- If A are designed to evaluate some inherited attributes, then the depended attributes are passed via parameters.
- If a caller's synthesized attribute requires some attributes of A , then these attributes are wrapped as the return value that will be received by the caller.

Syntax-Directed Translation

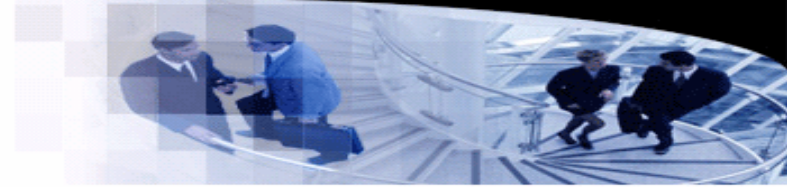
- Translation via recursive descent parsers

- A attribute grammar of C-style declarations
 - Decl: declaration
 - S: declaration specifier
 - L: declarator list
 - R: declarator
 - P: pointer
 - D: direct declarator
- The “type” attributes are inherited.
- The “*P.len*” attribute is synthesized.
- The evaluation rules are designed to pass some “type” attribute to each identifier, so the type of each identifier is



Productions	Semantic Rules
$Decl \rightarrow SL$	$L.type = S.type$
$S \rightarrow int$	$S.type = integer$
$L \rightarrow R, L^1$	$L^1.type = L.type$ $R.type = L.type$
$L \rightarrow R$	$R.type = L.type$
$R \rightarrow P D$	$D.type$ $= pointer(D.type, P.len)$
$R \rightarrow D$	$D.type = R.type$
$P \rightarrow^* P^1$	$P.len = P^1 + 1$
$P \rightarrow^*$	$P.len = 1$
$D \rightarrow id$	$id.type = D.type$
$D \rightarrow D^1 []$	$D^1.type = array(D.type)$
$D \rightarrow D^1 ()$	$D^1.type = function(D.type)$
$D \rightarrow (R)$	$R.type = D.type$

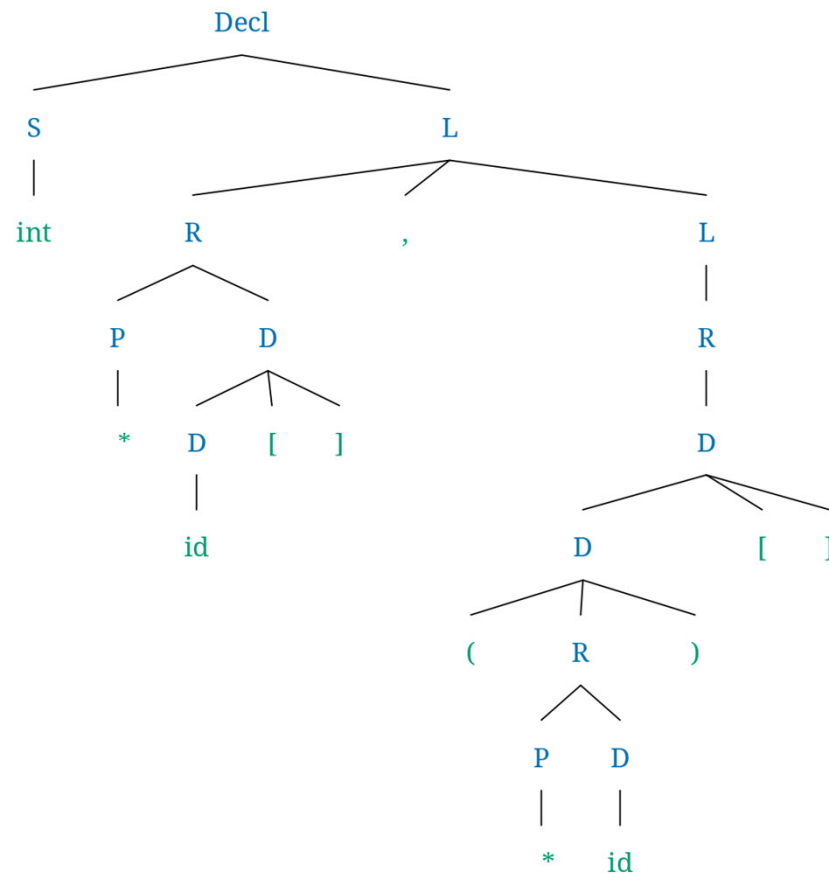
Syntax-Directed Translation



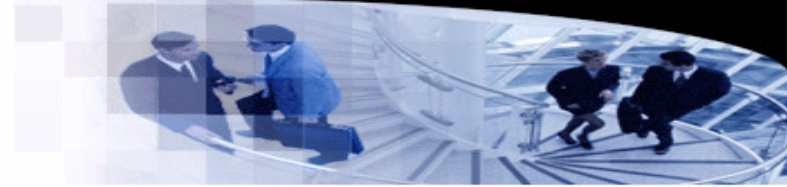
- Translation via recursive descent parsers

- Example:

- `int *p[], (*p)[]`



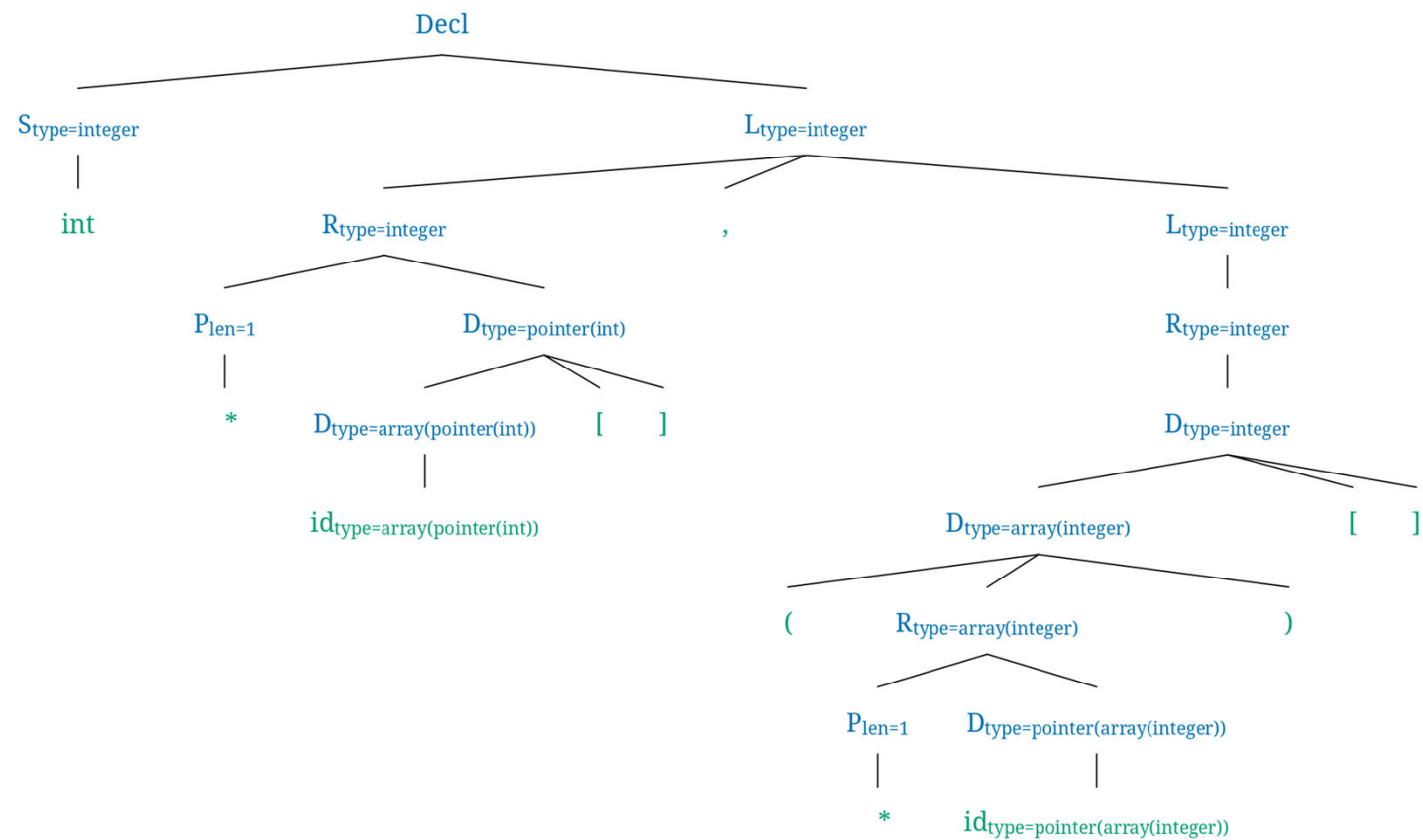
Syntax-Directed Translation



- Translation via recursive descent parsers

- Example:

- `int *p[], (*p)[]`



Syntax-Directed Translation

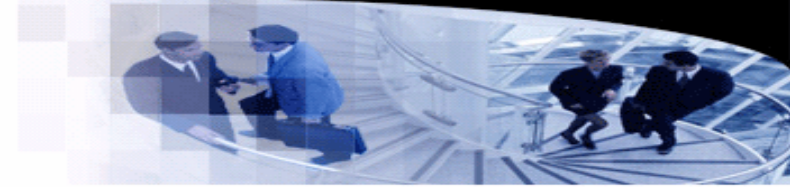
- Translation via recursive descent parsers
 - A pure functional recursive descent parser.
 - Define tokens and type.

```
type token =  
  | Identifier  
  | Integer  
  | Char  
  | Symbol of char;;
```

```
type tp =  
  | TUnknown  
  | TInt  
  | TChar  
  | TArray of tp  
  | TPtr of tp;;
```

```
tp -> unit  
let rec print_type t =  
  match t with  
  | TInt -> print_string "integer"  
  | TChar -> print_string "character"  
  | TUnknown -> print_string "()  
  | TArray(s) -> print_string "[]->"; print_type s  
  | TPtr(s) -> print_string "*->"; print_type s  
;;  
  
int -> tp -> tp  
let rec pointer len t =  
  if len <= 1 then TPtr(t) else TPtr(pointer (len - 1) t);;
```

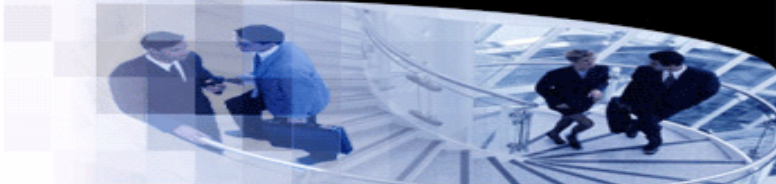
- The primitive types are TInt (integer) and TChar (character).
- The complex types are TArray (array) and TPtr (pointer).



Syntax-Directed Translation

- Translation via recursive descent parsers

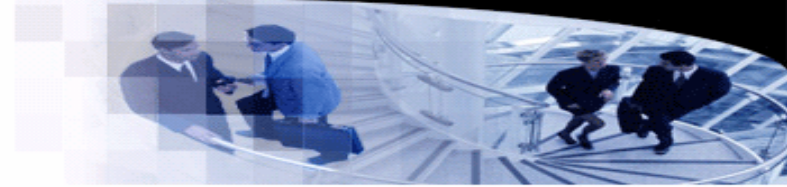
- $Decl \rightarrow SL$: Design the function `parseDecl` to parse *Decl*.
- Invoke `parseS` to get the synthesized “type” attribute, and pass the attribute to `parseL` to calculate an inherited attribute.
- Most functions like `parseL` would return a list of tokens to represent the current input, as this parser is pure functional.



```
(* Decl -> S L *)
token list -> unit
let parseDecl input =
  match input with
  | [] -> ()
  | tok::xs ->
    match tok with
    | Integer | Char ->
      let t = parseS tok in
      parseL t xs
  | _ -> ()
;;
```

```
token -> tp
let parseS tok =
  match tok with
  | Integer -> TInt
  | Char -> TChar
  | _ -> raise (Failure "Error S")
;;
```

Syntax-Directed Translation



- Translation via recursive descent parsers

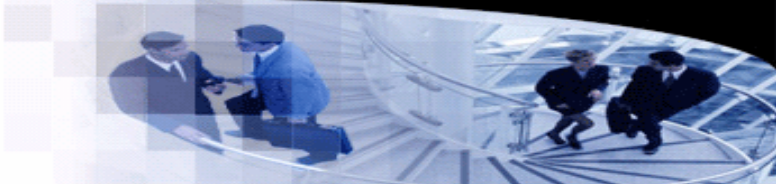
- $L \rightarrow R, L | \epsilon$
- It takes $\mathbf{t}$ as the current inherited type.
- $\mathbf{input}$ represents the remaining tokens to parse.
- The type is passed to `parseR` and `parseL`.
- Exercise:
 - This function does not check the *Follow* function.
 - Try to fix this issue.

```
(* L -> R, L | epsilon *)
tp -> token list -> unit
let rec parseL t input =
  match input with
  | [] -> ()
  | tok::xs ->
    match tok with
    | Symbol('(') | Symbol('*') | Identifier ->
      let s = parseR t input in
      (match s with
       | Symbol(',')::ss -> parseL t ss
       | _ -> ())
    | _ -> raise (Failure "Error L")
```

Syntax-Directed Translation

- Translation via recursive descent parsers

- $R \rightarrow P D \mid D$
- `parseP` is used to count the number of “*”s. The result is in the return value as a synthesized attribute.
- When $R \rightarrow P D$ is selected, it first invoke `parseP` to calculate the number of pointers.
- Then it constructs a new type by adding pointer constructors to `t`.
- The constructed new type is passed to `parseD`.



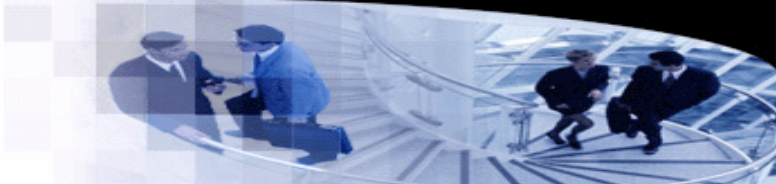
```
(* R -> P D | D *)
tp -> token list -> token list
and parseR t input =
  match input with
  | Symbol('*')::xs ->
    let (len, s) = parseP 0 input in
    parseD (pointer len t) s
  | Symbol('(')::xs | Identifier::xs ->
    parseD t input
  | _ -> raise (Failure "Error R")
```

```
(* P -> *P | * *)
int -> token list -> int * token list
and parseP len input =
  match input with
  | Symbol('*')::xs -> parseP (len + 1) xs
  | Symbol('(')::xs | Identifier::xs -> (len, input)
  | _ -> raise (Failure "Error P")
```

Syntax-Directed Translation

- Translation via recursive descent parsers

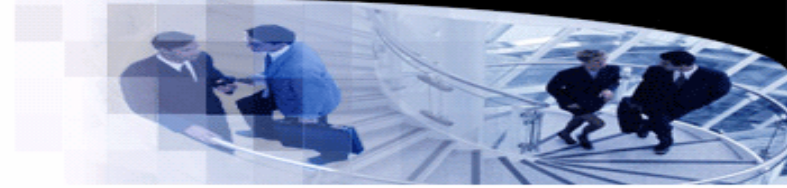
- The original productions of D are left-recursive.
- We obtain the following new productions by eliminating left recursions.
 - $D \rightarrow id\ D' \mid (R)$
 - $D' \rightarrow []\ D' \mid \epsilon$
- `parseD1` is used to add postfix types to the current type.
- Once $D \rightarrow id\ D'$ is successfully derived, we know the type of `id`, and therefore print the type.



```
(* D -> id D' | (R) *)
tp -> token list -> token list
and parseD t input =
  match input with
  | Identifier::xs ->
    let (ts, s) = parseD1 t xs in
    print_type ts; print_endline " "; s
  | Symbol('(')::xs ->
    let s = parseR t xs in
    (match s with
     | Symbol('')::ss -> ss
     | _ -> raise (Failure "Error (R)"))
  | _ -> raise (Failure "Error D")

(* D' -> [] D' | epsilon *)
tp -> token list -> tp * token list
and parseD1 t input =
  match input with
  | Symbol '['::Symbol(']')::xs ->
    let (ts, s) = parseD1 t xs in
    (TArray(ts), s)
  | Symbol('')::xs | Symbol('$')::xs | Symbol(', ')::xs -> (t, input)
  | _ -> raise (Failure "Error D1")
;;
```


Syntax-Directed Translation



- Translation via recursive descent parsers
 - Wrap them together.
 - Now we can use a more convenient input to test the parser.
- Exercise:
 - The parser fails to parse inputs with parenthesis.
 - Fix this issue by your self.
 - Hint:
 - Let $L \rightarrow L, R \mid R$ decide the final type.

```
string -> unit
let parse input =
  let explode s = List.init (String.length s) (String.get s) in
  let m = fun x ->
    (match x with
    | 'i' -> Integer
    | 'a' -> Identifier
    | _ -> Symbol(x)) in
  let l = List.map m (explode input) in
  parseDecl l;;

parse "i*a[]$";;
```